

To: Charalampos Mainas (@cmainas) & Anastassios Nanos (@ananos) — Nubificus / urunc

14 May 2026

Project: CNCF · urunc · Improve lifecycle management of sandbox monitors

Upstream: urunc-dev/urunc#112

Dear Charalampos and Anastassios,

I am writing in support of my application for the LFX Mentorship 2026 Term 2 project **CNCF / urunc — Improve lifecycle management of sandbox monitors** (upstream issue [urunc-dev/urunc#112](https://github.com/urunc-dev/urunc/issues/112)). The four sections below answer the cover-letter questions LFX asks for, with the full design sketch, weekly plan, and prior work folded in.

01 How did you find out about the mentorship program?

I follow CNCF projects on GitHub, and while reading through the 2026 Term 2 project ideas list I saw the urunc entries. urunc itself I had already been running locally for a few weeks — the unikernel-as-container model is what I wanted to understand at depth, and a few weeks ago I filed [urunc-dev/urunc#389](https://github.com/urunc-dev/urunc/issues/389) against it (high host CPU caused by console output). That informal contribution is where I first interacted with you both. The mentorship gave me a clean way to extend that work into something more sustained and aligned with the project's roadmap, rather than continuing as one-off issues.

02 Why are you interested in this program — and this project specifically?

The program

I want to spend three months working on a single piece of cloud-native infrastructure at depth, with maintainers reviewing my code, instead of the scattered drive-by pattern that one-off OSS contributions tend to settle into. urunc is the right *size* of project for that — small enough that I can hold the whole codebase in my head, load-bearing enough that the work matters. Three months of dense feedback from the people who actually own the codebase is the fastest learning loop I can think of.

The problem in this project

urunc today shells out to QEMU or Firecracker with a long `-x ... -y ...` command string. That is enough to *start* a sandbox, but everything after that — checking what the guest is doing, hotplugging a device, asking the monitor to shut down gracefully, attaching a console after the fact — is either impossible or has to be approximated by signalling the host process and hoping for the best.

Both QEMU and Firecracker already expose a much better interface for this: QEMU has the QEMU Machine Protocol (QMP, line-delimited JSON over a Unix socket), and Firecracker exposes a REST-style API socket. Issue #112 is exactly this gap — urunc has CLI for spawn but no socket-based control plane for the lifecycle that follows. The cost of not having that surface shows up in real places: graceful pause/resume isn't really possible, hotplug isn't really possible, and the urunc-side state machine has to infer monitor state from process exit codes instead of asking the monitor directly. It also means each

new monitor we add (cloud-hypervisor, Dragonball, etc.) is a fresh integration with no shared abstraction.

My proposed approach

I see this project as two pieces that have to land together. The first is a **monitor-agnostic control interface inside urunc** — an internal Go interface that captures the operations urunc actually needs (probe, query state, send shutdown, hotplug net/block, attach console). The existing `VMM` interface in `pkg/unikontainers/types/types.go` is a sensible home for this; right now it carries `BuildExecCmd`, `Stop`, `Path`, `UsesKVM`, `SupportsSharedfs`, `Ok`, and `PreExec` for HVT seccomp. The new surface extends it without breaking those. The second piece is the **per-monitor implementations** that translate the interface into QMP for QEMU and into the Firecracker API for Firecracker, plus a small fallback path for monitors that do not expose a socket (so the existing CLI-only flow keeps working).

Concretely, for QEMU I would generate `-qmp unix:<sock>,server,nowait` as part of the existing `BuildExecCmd` flow. After spawn, the sequence is: read the greeting, send `qmp_capabilities`, then any of the operational commands the interface exposes. The QMP commands I would implement in order of project priority are `query-status` (probe + state machine), `quit` (graceful shutdown to replace `killProcess`), `stop / cont` (pause/resume), `device_add / device_del` (hotplug for virtio-net and virtio-blk first), `chardev-add / chardev-remove` (console attach/detach), and `query-vcpus / query-block` for introspection. JSON over the socket is small enough that I'd rather hand-roll the wire types than pull in a heavy QMP library, but I will evaluate github.com/digitalocean/go-qemu and a couple of alternatives in the design doc and recommend one with reasons.

For Firecracker, the existing `firecracker-go-sdk` already wraps the API socket; the work is mostly wiring urunc's interface methods to `PauseVM` / `ResumeVM` / `PatchDrive` / `PatchMMDS` / `GetMachineConfig`, and deciding which subset we actually expose at the urunc layer. Firecracker's API surface is smaller than QEMU's, so the `MonitorControl` interface needs to be expressive enough for QMP without forcing Firecracker to fake operations it doesn't support — capability flags on the interface, the same shape urunc already uses for `SupportsSharedfs`.

Socket lifecycle is the tricky part: where the socket lives (per-container subdir under urunc's state dir is the natural choice), who cleans it up, and what happens when the monitor dies between us opening the socket and us using it. I will work the failure modes out in the design doc before writing code.

What I would explicitly not do

- I will not rip out the existing `BuildExecCmd` / `syscall.Exec` shape. The new socket-based control is layered on top, not a replacement.
- I will not also rewrite the seccomp handling (`PreExec` for HVT) or the monitor-rootfs work in the same project. Those are adjacent but big enough to be their own conversations.
- I will not add a new monitor (cloud-hypervisor, Dragonball, etc.) inside the LFX scope. The interface should make it easy to add one later — that's a different person's first PR.
- I will not introduce a new dependency for QMP unless the design doc justifies it.

What I would prove out before merging

- The QMP-controlled QEMU path can spawn, query state, gracefully shut down, and hotplug a virtio-net device end-to-end, with the existing nginx-on-unikraft integration tests still green.

- The Firecracker path can do the same subset.
- A simple benchmark — same workload as my #389 work, but measuring monitor-control overhead (number of round-trips, socket I/O cost) — so we have a real number for the design doc and not just "it works."
- The fallback path (CLI-only, no socket) still works for every current monitor.

03 What experience and skills do you have that are applicable?

Day job: low-level Linux infrastructure

I am a final-year B.Tech (Information Technology) student at IIITM Gwalior and currently a Software Developer intern at Juspay — India's largest payments orchestration platform. My day-to-day is the kind of work this project will need: I led the replacement of our CEPH block-storage tier with Lightbits NVMe/TCP (same workload at 48K TPS vs 37K TPS, on roughly a tenth of the hardware — 4 nodes vs 40 — with active-active Barbican HA on a shared backend). I shipped a thread-local and server-local cache layer in Rust on hot APIs that cut cross-node calls by ~4% at peak, and a Shamir-based secrets distribution layer with memory-only keys and Raft replication that removed our reliance on AWS KMS. The shape of that work — Go and Rust on Linux, syscalls, IPC, process management, Unix sockets — is exactly the shape this LFX project has.

Open source: shipped, not promised

Between November 2025 and January 2026 I worked as an open-source backend engineer on [Namma Yatri](#), the open-source ride-hailing platform that runs in production across India. My main piece there was [nammayatri/nammayatri#13442](#) — a distributed, event-driven payout scheduling system for special-zone rides, serving roughly 10,000 rides per day, which cut driver payout latency from weekly cycles down to T+2 hours. The interesting part was the Redis-backed delayed-job scheduler with per-ride locks and idempotent state transitions; that piece had to be exactly-once under production traffic without adding latency to the ride-booking hot path. The follow-up PR [#13452](#) surfaced the payout state to the driver app, and a separate [shared-kernel#1108](#) shipped a PaytmEDC payment-flow integration. All three are merged. I mention this because it is the closest thing I have to "shipped upstream code in a large unfamiliar Haskell codebase with maintainer review" — which is the muscle this LFX project will need.

Existing urunc work

Issue [urunc-dev/urunc#389](#) is mine. I wrote a small KVM-exit and throughput sampler (`bench-console.sh`) and traced the regression to per-byte PIO traps on the 16550 UART; the baseline `-serial stdio` configuration was costing roughly 1.16 M PIO exits per 5-second window while a simple virtio-console swap drops that to ~0 and lifts throughput from 2,016 to 73,378 lines/sec (about 36×). That thread shaped how I think about urunc's monitor surface and is part of why this project is the one I want to work on.

Other relevant background

- **BREAK OS (kernel from scratch in C++)**. A small 32-bit i386 monolithic kernel — bootstrap, protected mode, PIC + ISR/IRQ wiring, 64KB GDT, ~0.1 ms interrupt response. This LFX project is essentially the userspace side of the same world.

- **PostgreSQL on Kubernetes / OpenStack.** A 5-layer nested virtualization stack (KVM → OpenStack → K8s → Postgres) on an i9-14900 with NUMA-aware CPU pinning and 16 GB hugepages; zero-touch HA provisioning via Cluster API (CAPO) + CloudNativePG, ~12 min cluster bring-up, ~7 s automatic failover, RPO=0. Not directly urunc, but the same shape — API-driven lifecycle of stateful infrastructure on top of VMM-based isolation.
- **VMM familiarity.** My entire urunc lab runs QEMU on KVM; I have used Firecracker through `firecracker-go-sdk` for a smaller side project; some `runc` reading plus the urunc debugging gave me the OCI runtime basics.
- **Empirical debugging posture.** The #389 work involved tracing per-byte PIO exits via `/sys/kernel/debug/kvm/io_exits` rather than guessing. I expect the QMP work to need the same posture, especially around socket-lifecycle edge cases.

Pre-application work (in flight before May 19)

Between now and the application deadline I am landing the following in the proposal repo so you can see the work, not just read the plan:

- A small standalone Go program that opens a Unix socket to a `qemu-system-x86_64` instance, sends `qmp_capabilities`, then `query-status` and `quit`. ~100 lines, intentionally throwaway — just to prove the wire shape and identify the rough edges.
- A second, equally small Go program against the Firecracker API socket using `firecracker-go-sdk`, doing `GetMachineConfig` and `PauseVM / ResumeVM` on a stopped microVM.
- A one-page sketch (`docs/design-sketch.md`) of what the `MonitorControl` interface signature would look like and where it would slot into `pkg/unikontainers/hypervisors/types.go`. Not a finished design — a starting point for the actual design doc in week 2.

This is not part of the LFX commitment; it is what I am doing this week regardless, because the only way to know whether the design is right is to write a little of it.

04 What do you hope to get out of this mentorship experience?

Three things, in priority order:

- **A real piece of upstream urunc that I designed and shipped end-to-end** — a monitor-control abstraction that survives review and that other contributors build on after me. The expected outcomes the project description lists (design document, implementation) become the visible artifacts.
- **Code-review feedback from you both at the volume you only get on a maintainer's project.** Reading reviews is how I learn fastest. Three months of dense feedback is the loop I am here for.
- **Enough fluency in urunc internals to keep contributing after the program ends.** I am not looking for a stipend and a goodbye. I am looking for the on-ramp to a project I expect to be involved in for a while.

The plan that gets us there: 12 weeks (8 Jun – 31 Aug 2026)

I treat the first two weeks as design-only and the last two as buffer plus write-up, which leaves eight weeks for code. I expect to break weeks 4–7 into multiple small PRs rather than one large one, so review can happen in parallel with development.

Wk	Dates	Focus	Concrete deliverable
1	Jun 8 – Jun 14	Onboarding + read	Re-read <code>pkg/unikontainers/hypervisors/</code> ; write a "current state of monitor lifecycle in urunc" gist for mentors to correct.
2	Jun 15 – Jun 21	Design doc, draft 1	Markdown design doc in <code>docs/</code> : <code>MonitorControl</code> interface, QMP + Firecracker mapping, socket lifecycle, fallback behaviour. Draft PR.
3	Jun 22 – Jun 28	Design doc, revisions	Address review; settle the interface signature and dependency decision. Tag the doc as ready once mentors sign off.
4	Jun 29 – Jul 5	QEMU/QMP MVP	Wire <code>-qmp</code> into QEMU <code>BuildExecCmd</code> ; implement capability handshake + <code>query-status</code> + <code>quit</code> . A urunc-launched sandbox can be queried and gracefully stopped over QMP.
5	Jul 6 – Jul 12	QEMU full lifecycle	<code>cont / stop</code> (pause/resume), <code>device_add / device_del</code> for virtio-net and virtio-blk, error handling for unsupported commands. Unit tests for the wire codec.
6	Jul 13 – Jul 19	Firecracker API	Implement the same <code>MonitorControl</code> surface against <code>firecracker-go-sdk</code> . Pause/resume, drive patch, network patch where applicable.
7	Jul 20 – Jul 26	Integration	Plumb the control path through <code>unikontainers.go</code> so urunc's state machine uses it. Confirm CLI-only fallback still works for monitors without sockets.
8	Jul 27 – Aug 2	Tests + CI	Integration tests in the existing layout. Extend <code>bench-console.sh</code> to time QMP round-trip cost. CI workflow update.
9	Aug 3 – Aug 9	Hotplug demo	Start a unikraft-nginx sandbox, hotplug a second virtio-net device, verify connectivity. Document the recipe.
10	Aug 10 – Aug 16	Bug-fix + docs	Review-comment cleanup. User-facing docs: how to talk to a urunc sandbox's QMP socket, how to opt into the control path.
11	Aug 17 – Aug 23	Buffer	Reserved for slippage. If none, start a follow-up issue from the mentors' backlog.
12	Aug 24 – Aug 31	Wrap-up + blog	Final PR clean-up. Mentee blog post for the CNCF / Nubificus blog covering the architecture and the numbers.

Availability

I graduate from IIITM Gwalior in May 2026, so the LFX program (8 Jun – 31 Aug 2026) sits cleanly between graduation and any later commitments. My current Juspay internship ends in May. I am committing to the program as a full-time effort (40 hours/week) — I will not be juggling other engagements during it. Time zone is IST (UTC+5:30); I overlap with Athens (UTC+2/+3) for roughly 5 hours every working day, which I have already been doing for the #389 thread.

Thank you for considering my application. I am happy to follow up on anything in the proposal repo — particularly the design sketch, once it is up — or to walk through any part of the plan in more detail.

Warm regards,

Raman Kapoor

B.Tech (Information Technology), ABV-IIITM Gwalior · Graduating May 2026

Full proposal repository: github.com/unchangedraman/proposal